



DSY1102 - Desarrollo Orientado a Objetos
Encapsulamiento en Java



¿Qué aprenderemos hoy?

01

¿Qué es el encapsulamiento?

02

¿Por que es importante el encapsulamiento?

03

Modificadores de acceso

04

Implementación del encapsulamiento

05

Getters y Setters

06

Validación en Setters

07

Encapsulamiento en Java

08

Conceptos relacionados

09

Inmutabilidad y encapsulamiento defensivo

10

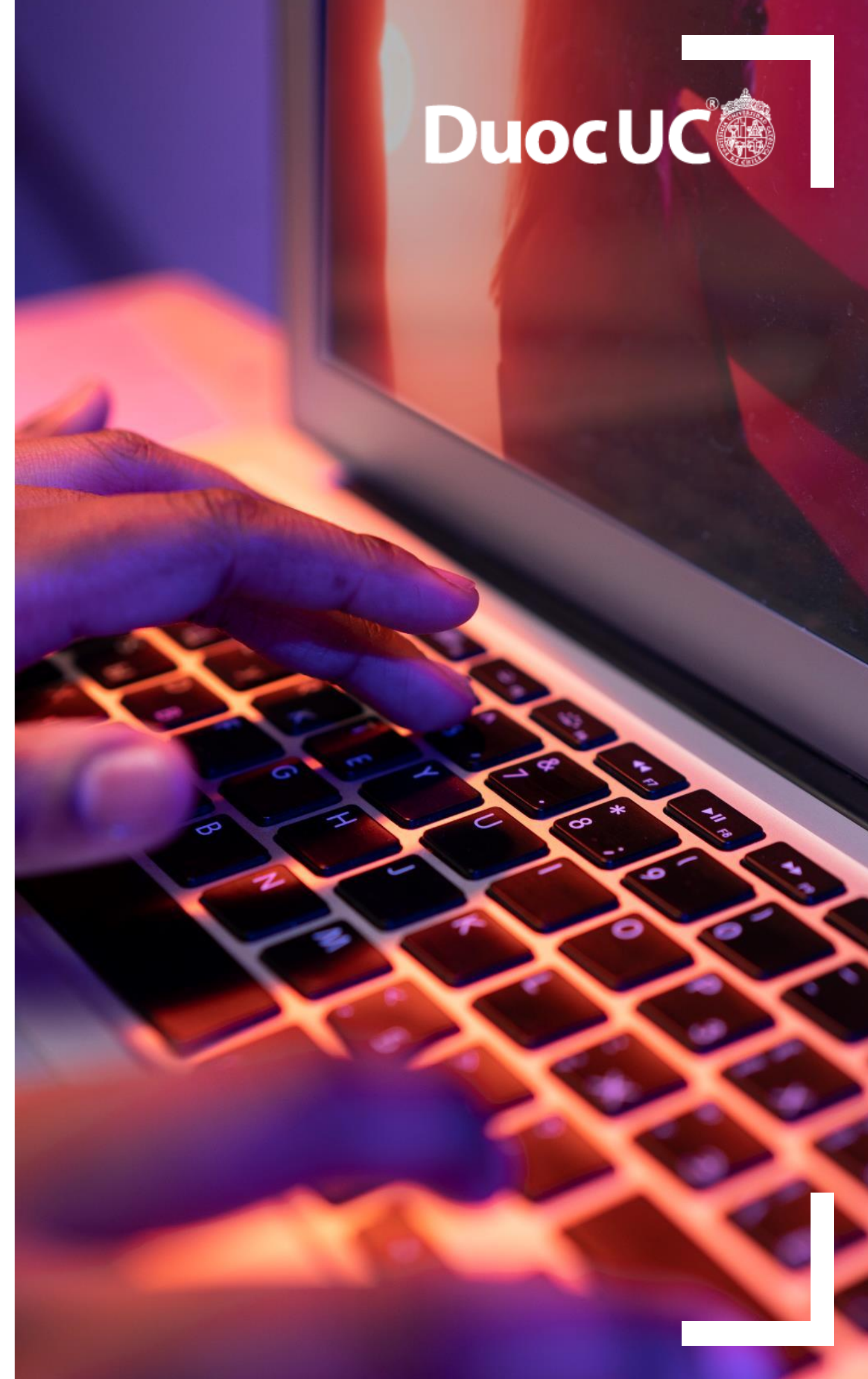
Buenas practicas

11

Errores Comunes

12

Checklist de encapsulamiento



</> Programación Orientada a Objetos

Java

Herencia en Java

Reutilización, especialización y polimorfismo con ejemplos de animales.



Reutilización
Evita código duplicado



Especialización
Añade nuevo comportamiento

```
Perro.java

public class Animal {

    public void hacerSonido () {
        System.out.println("...");
    }

    public class Perro extends Animal {

        @Override
        public void hacerSonido() {
            System.out.println("Guau!");
        }

    }

}
```





Programación Orientada a Objetos

Java

Encapsulamiento en Java

Protegiendo datos y diseñando clases robustas
con el principio de ocultamiento de información



Privacidad
Datos protegidos



Control
Acceso regulado

```
Persona.java

public class Persona {

    private String nombre;
    private int edad;

    public String getNombre() {

        return this.nombre;
    }

    public void setNombre(String nombre) {

        this.nombre = nombre;
    }
}
```



¿Qué es el encapsulamiento?

Ocultamiento del estado interno y exposición controlada

💡 Definición y Claves

Es el principio de la POO que consiste en **ocultar el estado interno** de un objeto y exponer solo una **interfaz controlada**.



Protección de datos

Evita manipulaciones indebidas del estado interno de los objetos.



Control de acceso

Regula quién y cómo se pueden modificar los datos mediante métodos.



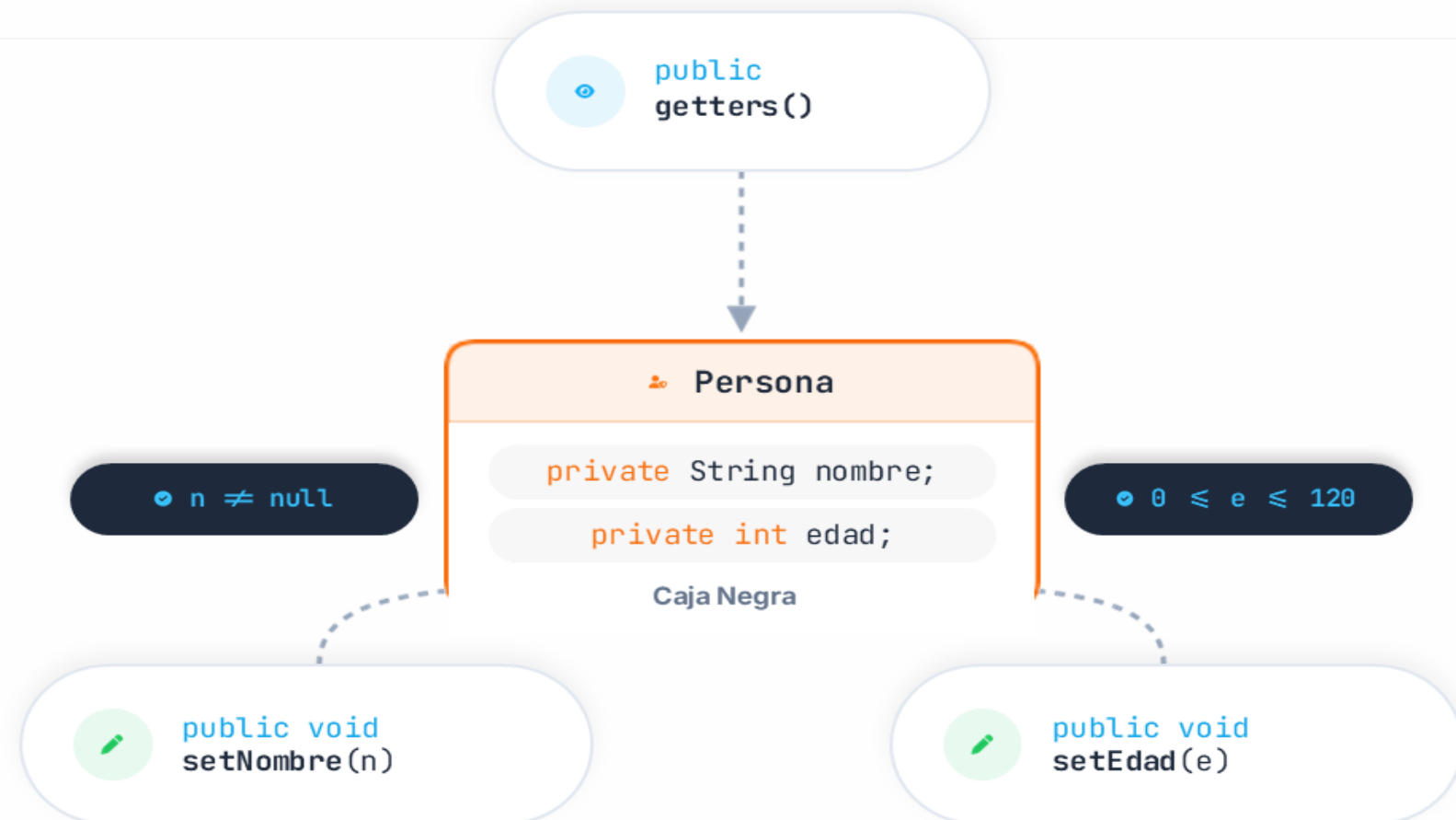
Mantenibilidad

Permite realizar cambios internos sin romper el código de los clientes.

Objetivo Final:

- ✔ Clases seguras y comportamiento predecible

Flujo de Acceso a una Clase Encapsulada



¿Por qué es importante el encapsulamiento?

Protección de estado y acceso regulado mediante métodos

★ Beneficios Principales

El encapsulamiento convierte a los objetos en **cajas negras** seguras, exponiendo solo lo necesario.



Proteger estado interno

Evita modificaciones inválidas o accidentales desde otras partes del código.



Reducir acoplamiento

El código cliente no depende de la estructura interna de los datos, solo de la interfaz.



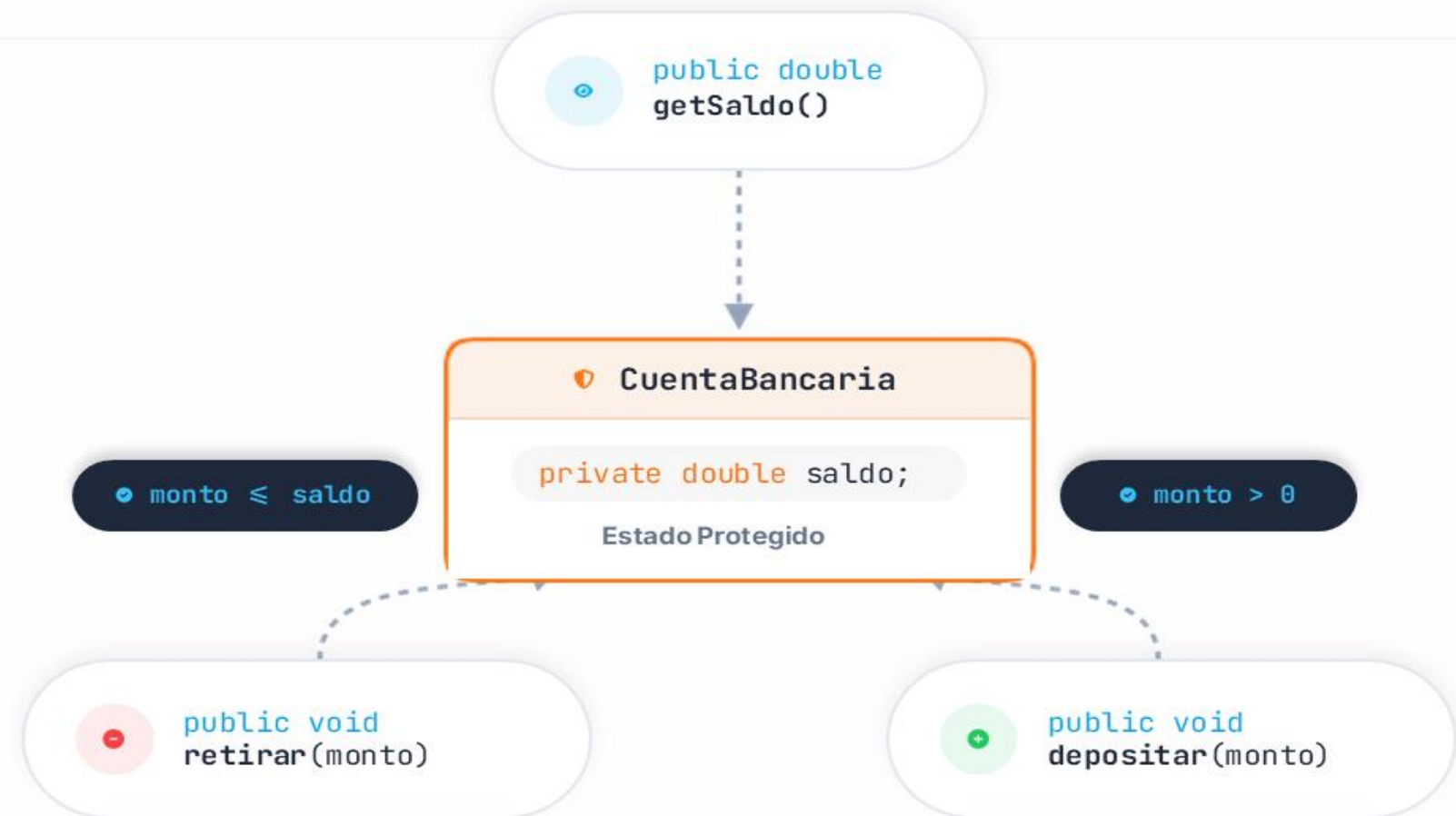
Validación centralizada

Las reglas de negocio y restricciones se aplican en un único lugar (los setters o métodos).

Resultado Final:

- Software robusto y mantenible

Arquitectura de Acceso Controlado





Modificadores de Acceso

Control de visibilidad en Java

Niveles de Acceso

Los modificadores definen qué componentes pueden interactuar con los miembros de una clase.



private

Oculto la implementación interna, visible solo en la clase actual.



(default)

Acceso de paquete: visible para clases dentro del mismo paquete.



protected

Herencia: extiende el acceso a subclases en otros paquetes.



public

API Pública: expone el contrato al mundo exterior sin restricciones.

Regla de Oro:

💡 Empieza con private y abre según necesidad

Escala de Visibilidad

Más visible

Menos visible



public

Cualquier clase en el proyecto



protected

Mismo paquete + Subclases



(default)

Solo en el mismo paquete



private

Misma clase

 Menos restrictivo (public)

 Más restrictivo (private)





Implementación del Encapsulamiento

Flujo coordinado entre métodos, validaciones y estado interno



Componentes del Flujo

Una clase bien encapsulada no solo usa **private**, sino que coordina varios componentes para proteger su integridad.



1. Interfaz Pública

Punto de acceso único para que los clientes interactúen.



2. Filtro de Validación

Reglas de negocio aplicadas antes de realizar cambios.



3. Estado Protegido

Datos ocultos y seguros frente a mutaciones externas.



4. Lógica Oculta

Métodos auxiliares privados que soportan la operación.

Resultado:

✓ Alta cohesión y bajo acoplamiento

Flujo de Ejecución Interna



Métodos Públicos

```
public void setEdad(int e)
```

Valida



Filtro de Validación

```
if (e > 0) { ... }
```

Delega

Asigna



Lógica Interna

```
private void calc()
```

Usa



Atributos Privados

```
private int  
edad;
```





Getters y Setters: Acceso Controlado

Patrón de acceso a datos con validación



Control de Acceso

Los **getters** exponen lectura controlada y los **setters** permiten modificar el estado aplicando reglas de validación.



Validar datos

Verifica entradas antes de asignar valores, rechazando información incorrecta.



Evitar cambios inválidos

Impide transiciones a estados que rompan la consistencia del objeto.



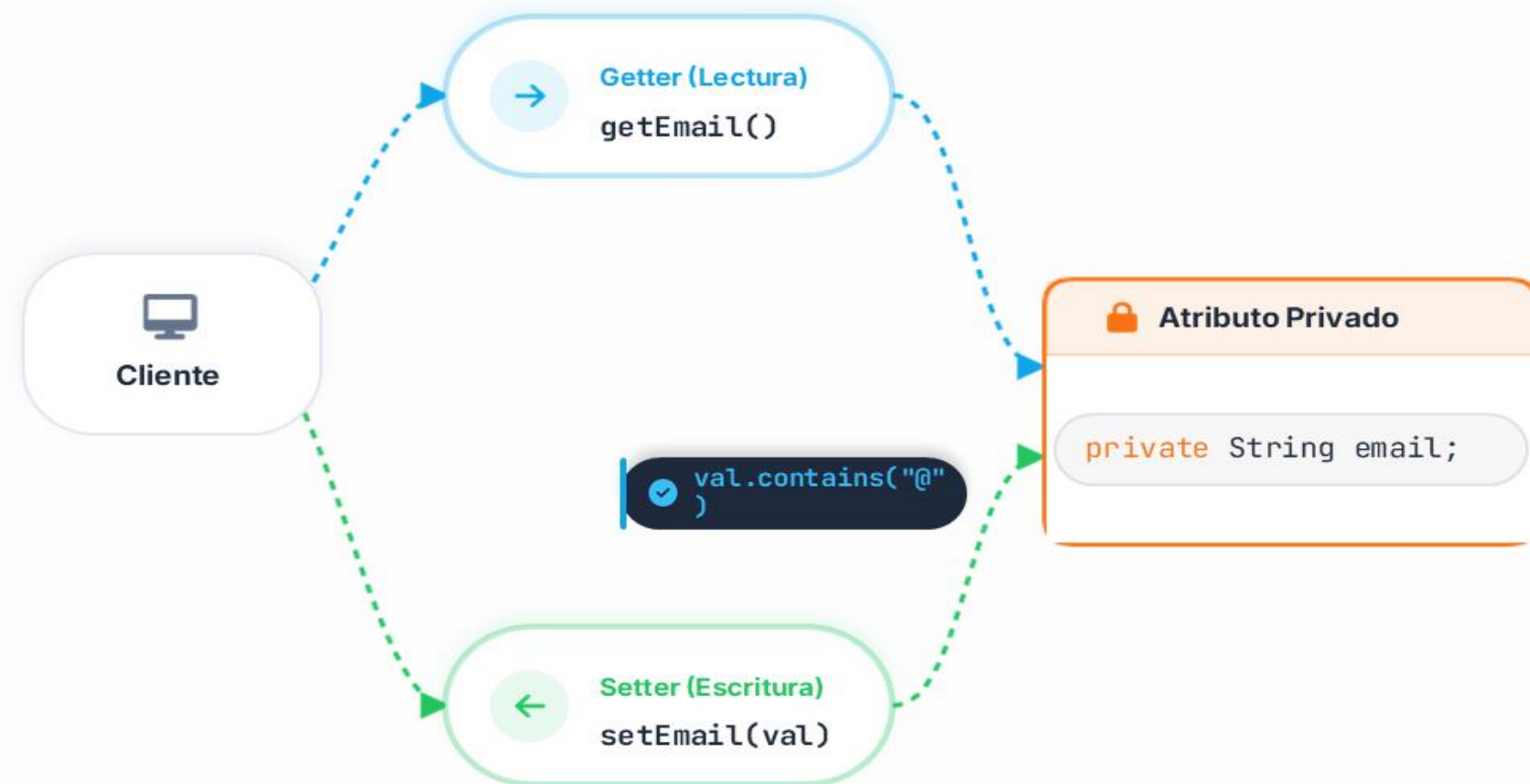
Mantener invariantes

Preservar reglas de negocio estrictas durante todo el ciclo de vida del objeto.

El Patrón Clave:

 Atributos ocultos, métodos expuestos

Flujo de Acceso a Datos



Encapsulamiento



Validación Activa





Validación en Setters

Controlando el estado de los objetos con reglas de negocio

★ Beneficios de Validar

Los setters no solo asignan valores, también actúan como **guardianes** del estado interno de los objetos.



Integridad de datos

Evita estados inválidos o absurdos (ej. edad negativa) que corrompan el objeto.



Centralización

Las reglas de negocio se aplican en un único lugar, facilitando el mantenimiento.



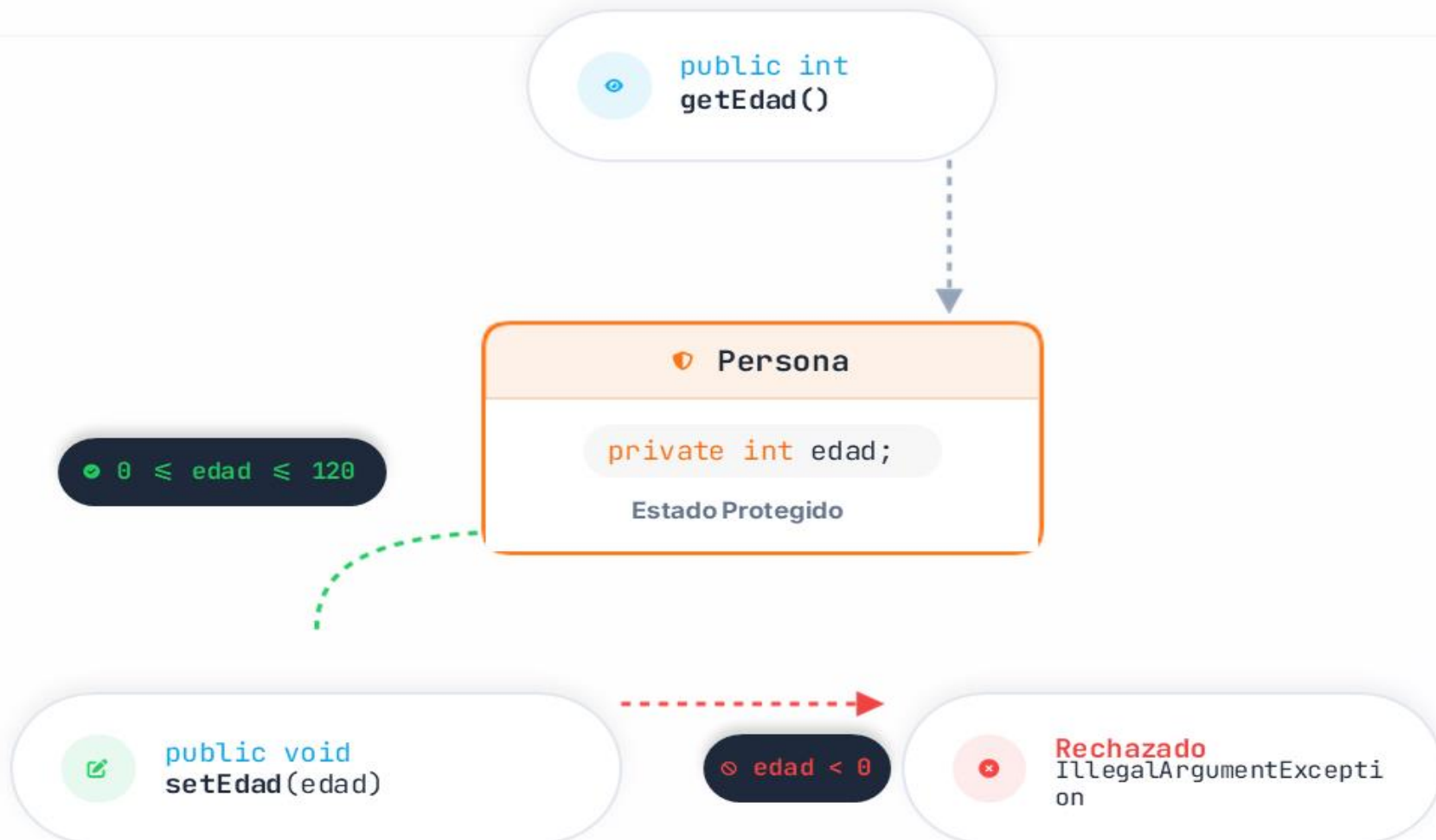
Seguridad del sistema

Protege la aplicación de fallos provocados por entradas incorrectas de otras capas.

Resultado Final:

- ✔ Objetos siempre consistentes

Arquitectura de Validación



Encapsulamiento en Java

CuentaBancaria.java

Implementación completa con encapsulamiento

```
public class CuentaBancaria {  
    private double saldo = 0.0;  
    // Constructor  
    public CuentaBancaria(double saldoInicial) {  
        This.saldo = saldoInicial;  
    }  
    // Getters y Setters  
    public double getSaldo() {  
        return saldo;  
    }  
    public void depositar(double monto) {  
        if (monto > 0) {  
            saldo += monto;  
        }  
    }  
    public boolean retirar(double monto) {  
        if (monto > 0 && monto ≤ saldo) {  
            saldo -= monto;  
            return true;  
        }  
        return false;  
    }  
}
```

 Los métodos controlan el acceso a los datos

Análisis del Código

Componentes clave del encapsulamiento

Ejemplo Práctico: Cuenta Bancaria



Campos Privados

El saldo está protegido con `private`



Validación

Verificación en métodos `depositar()` y `retirar()`



Encapsulamiento

Control de acceso al estado interno





Conceptos Relacionados

Encapsulamiento, Ocultación y Abstracción



Diferencias Claves

Aunque suelen confundirse, estos tres principios tienen propósitos distintos pero complementarios en el diseño de clases.



Abstracción

Se enfoca en el **diseño** (qué es y qué hace el objeto).



Encapsulamiento

Se enfoca en la **estructura** (cómo se agrupan datos y métodos).



Ocultación

Se enfoca en la **seguridad** (qué se esconde al exterior).

Relación Conceptual



Sinergia: La abstracción define el modelo esencial, el encapsulamiento organiza sus datos y métodos, y la ocultación restringe el acceso para protegerlos.



Inmutabilidad y Encapsulamiento Defensivo

Protegiendo colecciones internas y evitando efectos colaterales

Protección en Capas

El encapsulamiento defensivo **evita fugas de referencias** hacia estructuras mutables internas, garantizando que el objeto controle totalmente su ciclo de vida.



Creación Segura

Los constructores deben clonar los parámetros mutables antes de asignarlos.



Campos final y Copias

Definir el estado como inmutable y no proveer métodos setter.



Exposición Controlada

Devolver vistas inmutables o copias profundas en lugar de referencias directas.

Resultado Final:

Estado Estable e Inmutable

Flujo de Protección Defensiva



1. Creación Segura

Constructor con `copia profunda`

Evita referencias mutables



2. Estado Interno Protegido

Atributos `private final`

Evita fugas de estado



3. Métodos Seguros

Retornan `vistas inmutables`

✓ Resultado: Objeto 100% Estable



Buenas Prácticas de Encapsulamiento

Reglas de oro para un diseño orientado a objetos sólido

El Arte de Encapsular

Aplicar el encapsulamiento de manera disciplinada transforma un simple contenedor de datos en una **pieza de software robusta**.

No se trata solo de ocultar variables, sino de diseñar una **interfaz pública** que dicte cómo interactuar con el objeto de forma completamente segura.

Beneficios Clave:

- ✓ Garantiza alta cohesión
- ✓ Reduce el acoplamiento
- ✓ Facilita el mantenimiento

Mapa de Buenas Prácticas



⚠ Errores Comunes en Encapsulamiento

Antipatrones que rompen la protección del estado

⊘ Antipatrones a Evitar

Una clase mal encapsulada expone su estado interno, volviéndose frágil y difícil de mantener. Evita estas prácticas comunes.



Campos Públicos

Cualquier clase puede modificar los datos sin control ni reglas.



Setters sin Validación

Asignar valores ciegamente permite estados inválidos u objetos corruptos.



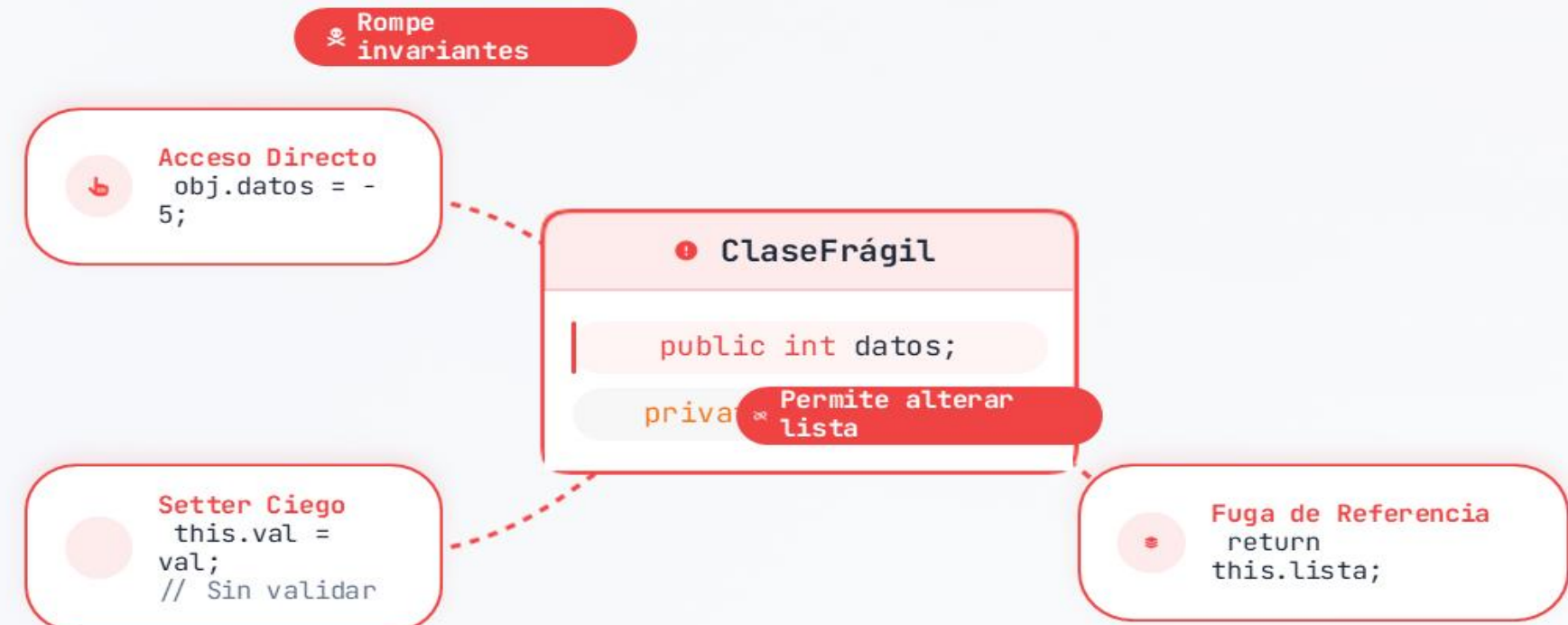
Devolver Colecciones Mutables

Permite que otras clases modifiquen listas internas sin pasar por los métodos.

Resultado Final:

- Código frágil y alta dependencia

Diagrama de Clase Frágil (Antipatrones)





Checklist de Encapsulamiento

Puntos clave para diseñar clases robustas y seguras

Reglas Esenciales

Verifica que tu clase cumpla con estas 4 reglas para garantizar un diseño sólido y fácil de mantener.



1. Atributos Private

Todo estado interno debe estar oculto al exterior.



2. Acceso Controlado

Expón solo métodos públicos con propósito claro.



3. Validación Efectiva

Verifica reglas de negocio antes de mutar el estado.



4. Consistencia

No expongas referencias directas a objetos mutables.

Resultado:

✓ 100% Encapsulamiento Logrado

Diagrama de Verificación



Consejo: Aplica este checklist mental cada vez que diseñes una nueva clase en Java.



Conclusiones y Próximos Pasos

Cierre del diseño orientado a objetos seguro

✓ Síntesis Final

El encapsulamiento protege el **estado interno**, refuerza **invariantes** y reduce drásticamente el acoplamiento entre componentes del sistema.



Empieza con private

Ocultar atributos por defecto.



Valida entradas

Usa setters como guardianes del estado.



Expón lo mínimo

Abre interfaces públicas solo si es necesario.

Resultado:

🏆 Arquitectura Robusta y Escalable

Flujo de Diseño Seguro



Proteger Datos

Estado interno seguro



Validar Cambios

Mantener invariantes



Diseñar APIs Claras

Exposición mínima

→ Próximos Pasos Recomendados

1

Inmutabilidad y Records

2

Patrones de Diseño

3

Módulos en Java



Felicitaciones
¡Has finalizado el módulo!

DuocUC[®]



CERCANÍA. LIDERAZGO. FUTURO.

7
AÑOS
ACREDITADO

 Comisión Nacional
de Acreditación
CNA-Chile

NIVEL DE EXCELENCIA
HASTA OCTUBRE 2031

Docencia de pregrado / Gestión institucional / Aseguramiento interno de la
calidad / Vinculación con el Medio / Investigación, creación y/o innovación